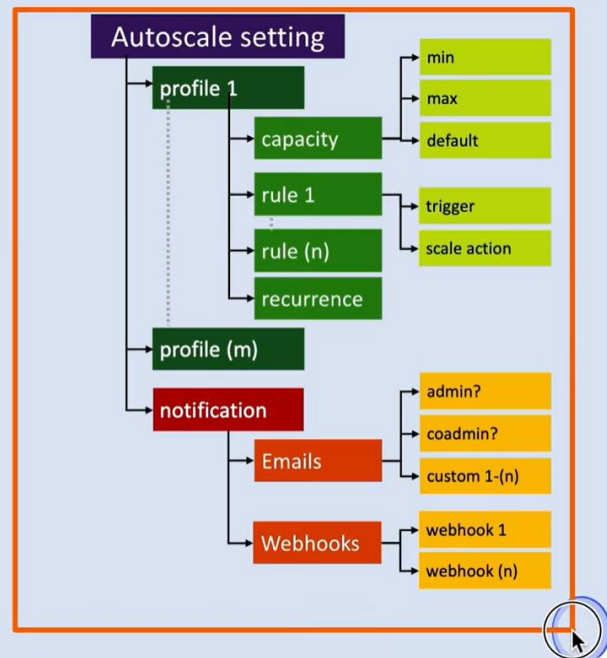


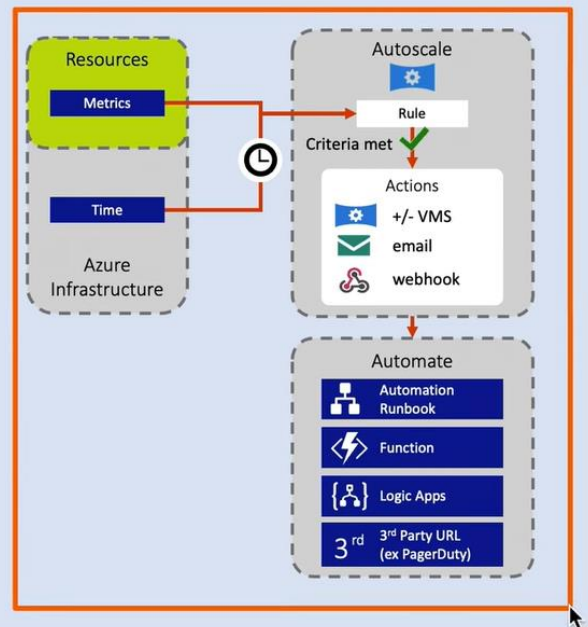
Autoscale hierarchy

- One autoscale setting
- Settings have one or more profiles
- Profiles have one or more rules:
 - Profiles can also have recurrences and capacity settings
- Notifications can be directly associated with an autoscale setting

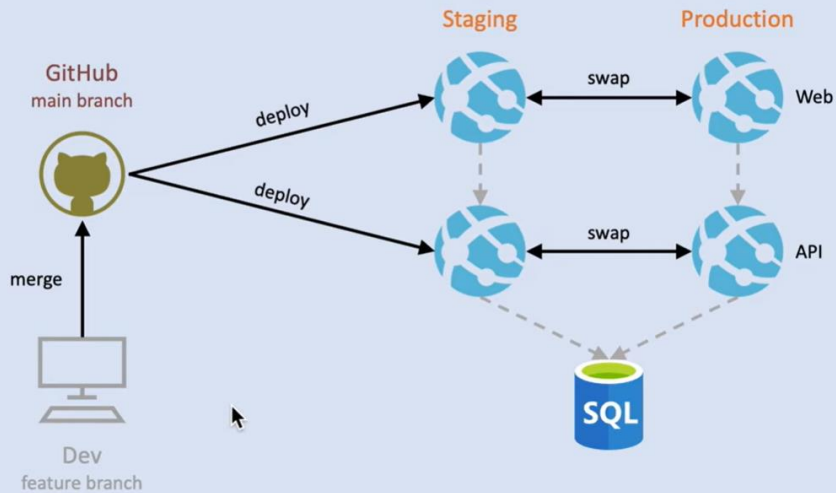


Autoscale workflow

1. Metrics are measured for a resource
2. When conditions are met (threshold surpassed), autoscale triggers:
 - a. Perform scale actions
 - b. Send notifications (alerts)
 - c. Send messages to webhooks for external automation



Modern deployment workflow



Automate slot management - Azure CLI

```
az webapp deployment slot create --name <appname> --resource-group <groupname> --slot staging
```

Create new slot

Swap slots

```
az webapp deployment slot swap --resource-group <groupname> --name <appname> --slot staging --target-slot production
```

Source

Target

Azure Functions

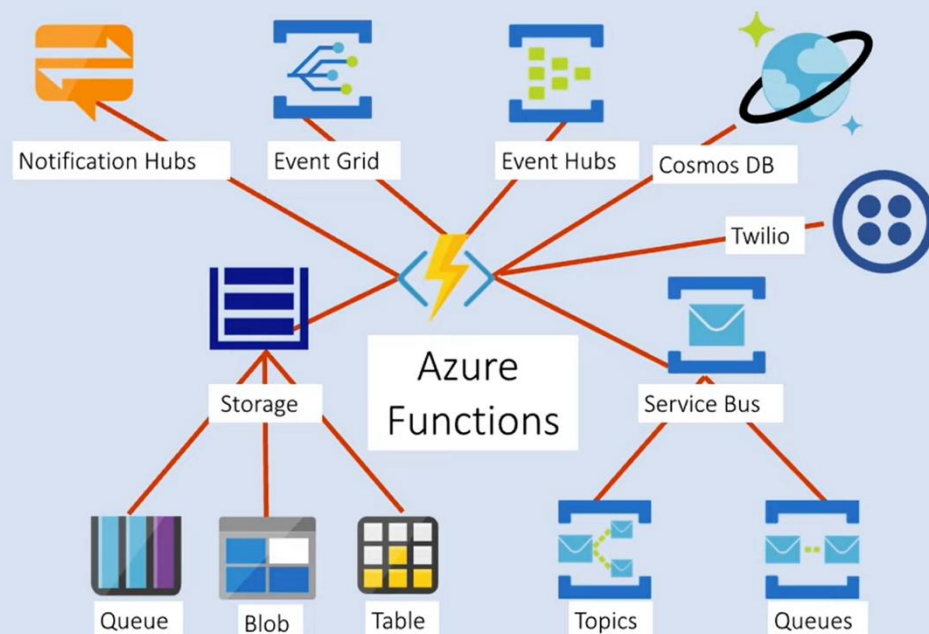
- Solution for running small pieces of code, or "functions," in the cloud:
 - Write only code that is relevant to business logic
 - Removes the necessity to write "plumbing" code to connect or host application components
- Build on open-source WebJobs code
- Supports a wide variety of programming languages, for instance:



- Even supports scripting languages, such as:



Function integrations



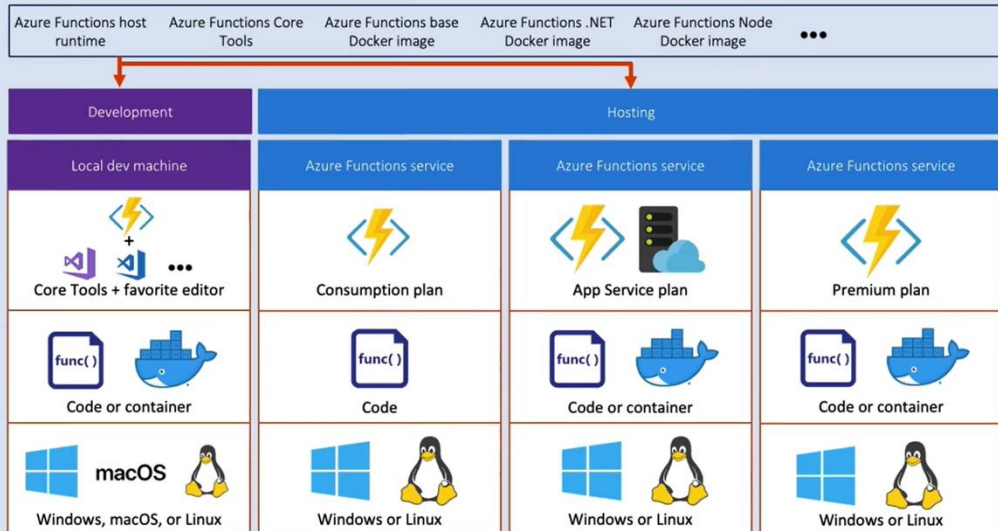
What can Azure Functions do?

- Run code based on HTTP requests
- Schedule code to run at predefined times
- Process new and modified:
 - Azure Cosmos DB documents
 - Azure Storage blobs
 - Azure Queue storage messages
- Respond to Azure Event Grid events by using subscriptions and filters
- Respond to high volumes of Azure Event Hubs events
- Respond to Azure Service Bus queue and topic messages

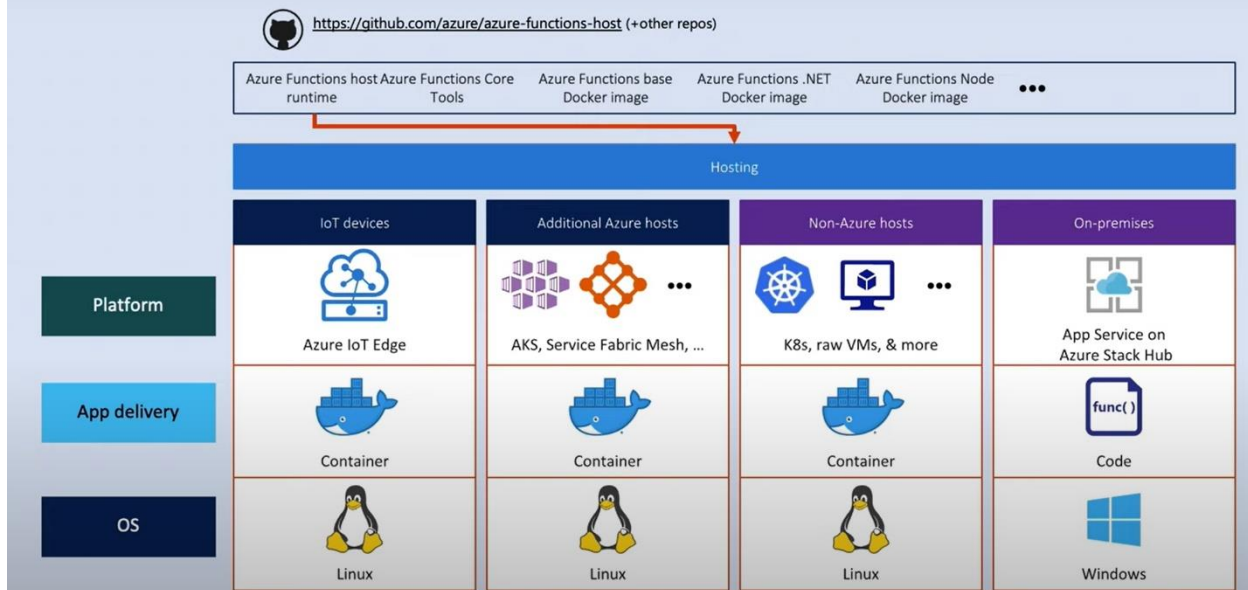
Azure Functions hosting



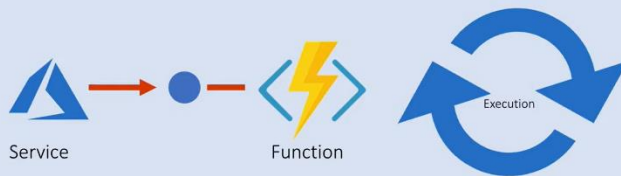
<https://github.com/azure/azure-functions-host> (+ other repos)



Azure Functions hosting (continued)

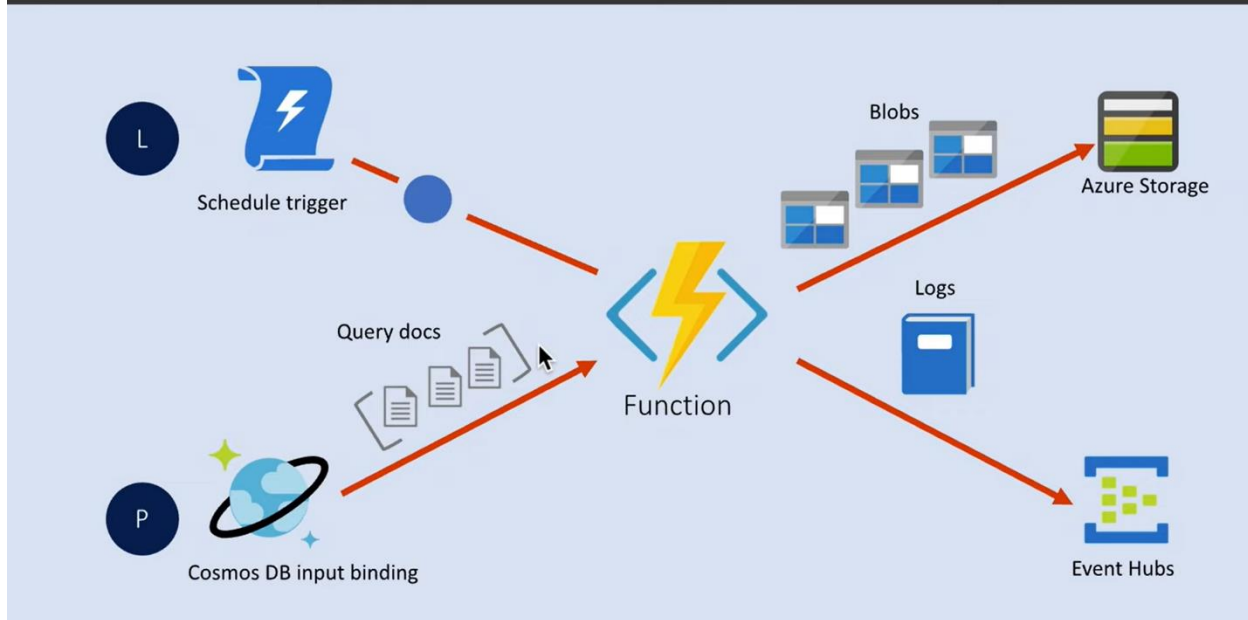


Triggers



- Triggers based on Azure services:
 - Cosmos DB
 - Blob and queues
 - Service Bus
 - Event Hub
- Triggers based on common scenarios:
 - HTTP request
 - Scheduled timer
- Triggers based on third-party services:
 - GitHub
- And more...

Trigger and Bindings example



Data added as Parameters

Azure Function (Java program – Function.java)

```
public class Function {  
    public String echo(  
        @HttpTrigger(  
            name = "request",  
            methods = {"post"},  
            authLevel = AuthorizationLevel.ANONYMOUS  
        )  
        String request, ExecutionContext context) {  
        return String.format(request);  
    }  
}
```



Azure Function (Python script – __init__.py)

```
import logging

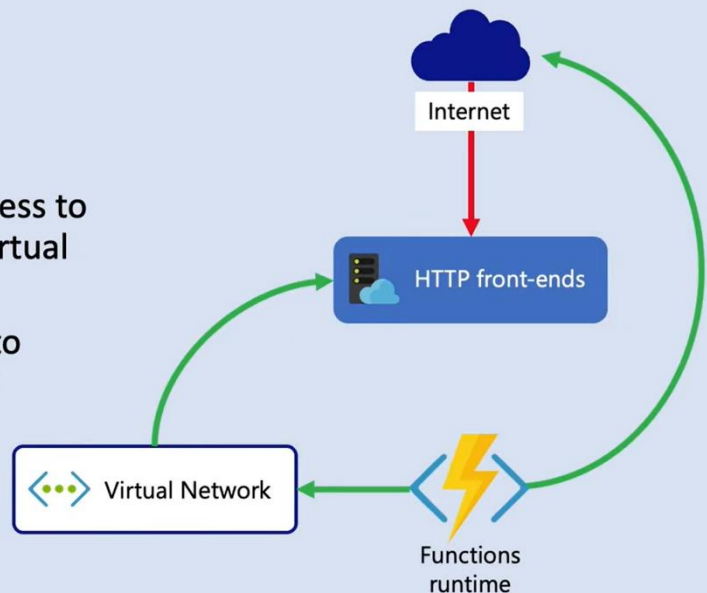
import azure.functions as func

def main(myblob: func.InputStream):
    logging.info(f"Python blob trigger function processed\n"
                f"Name: {myblob.name}\n"
                f"Blob Size: {myblob.length} bytes")
```

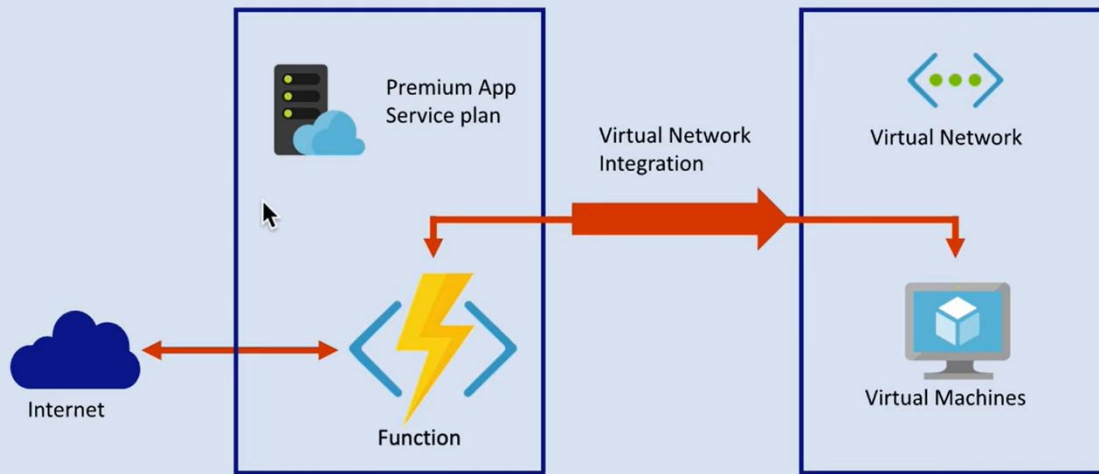


Integrating with Azure Virtual Network

- Requires the Premium plan
- Secures the inbound HTTP access to your app to one subnet in a Virtual Network
- Allows secure outbound calls to resources in a Virtual Network



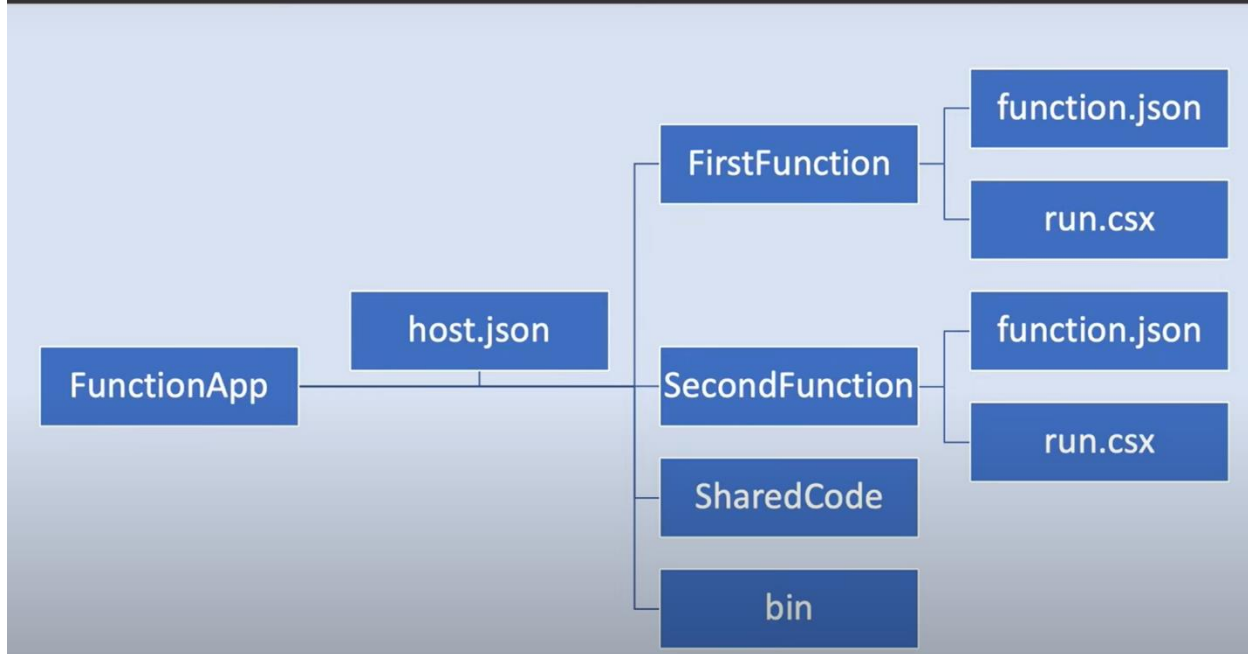
Azure Virtual Network integration example



Best practices

- Avoid long-running functions:
 - Functions that run for a long time can time out
- Use queues for cross-function communication:
 - If you require direct communication, consider Durable Functions or Azure Logic Apps
- Write stateless functions:
 - Functions should be stateless and idempotent
 - State data should be associated with your input and output payloads
- Code defensively:
 - Assume that your function might need to continue from a previous fail point

Function folder structure



Azure Functions in Visual Studio

- Visual Studio project type:
 - Develop, test, and deploy C# functions to Azure
 - Requires an **Azure development** workload installation
- Use WebJobs attributes to configure functions in C#
- Precompile C# functions:
 - Better cold-start performance
- Use the Azure Functions extension for Visual Studio Code to:
 - Build and run functions locally
 - Publish functions to Azure
 - Build C# pre-compiled class libraries
 - Build C# scripts by adjusting the extension settings
- Use the many built-in features and extensions for Visual Studio Code to make development easier

```

using System;
using Microsoft.Azure.WebJobs;
using Microsoft.Azure.WebJobs.Host;

namespace FunctionApp1
{
    public static class Function1
    {
        [FunctionName("QueueTriggerCSharp")]
        public static void Run([QueueTrigger("myqueue-items",
            Connection = "QueueStorage")]string myQueueItem, ILogger log)
        {
            log.Info($"C# Queue trigger function processed: {myQueueItem}");
        }
    }
}

```

Binding configuration

```

{
  "bindings": [
    {
      "name": "order",
      "type": "queueTrigger",
      "direction": "in",
      "queueName": "myqueue-items",
      "connection": "MY_STORAGE_ACCT_APP_SETTING"
    },
    {
      "name": "$return",
      "type": "table",
      "direction": "out",
      "tableName": "outTable",
      "connection": "MY_TABLE_STORAGE_ACCT_APP_SETTING"
    }
  ]
}

```

Note: An orange callout bubble points to the "name" field of the first binding, stating "Name of input parameter".

Binding-based code

```

#r "Newtonsoft.Json"

using Microsoft.Extensions.Logging;
using Newtonsoft.Json.Linq;

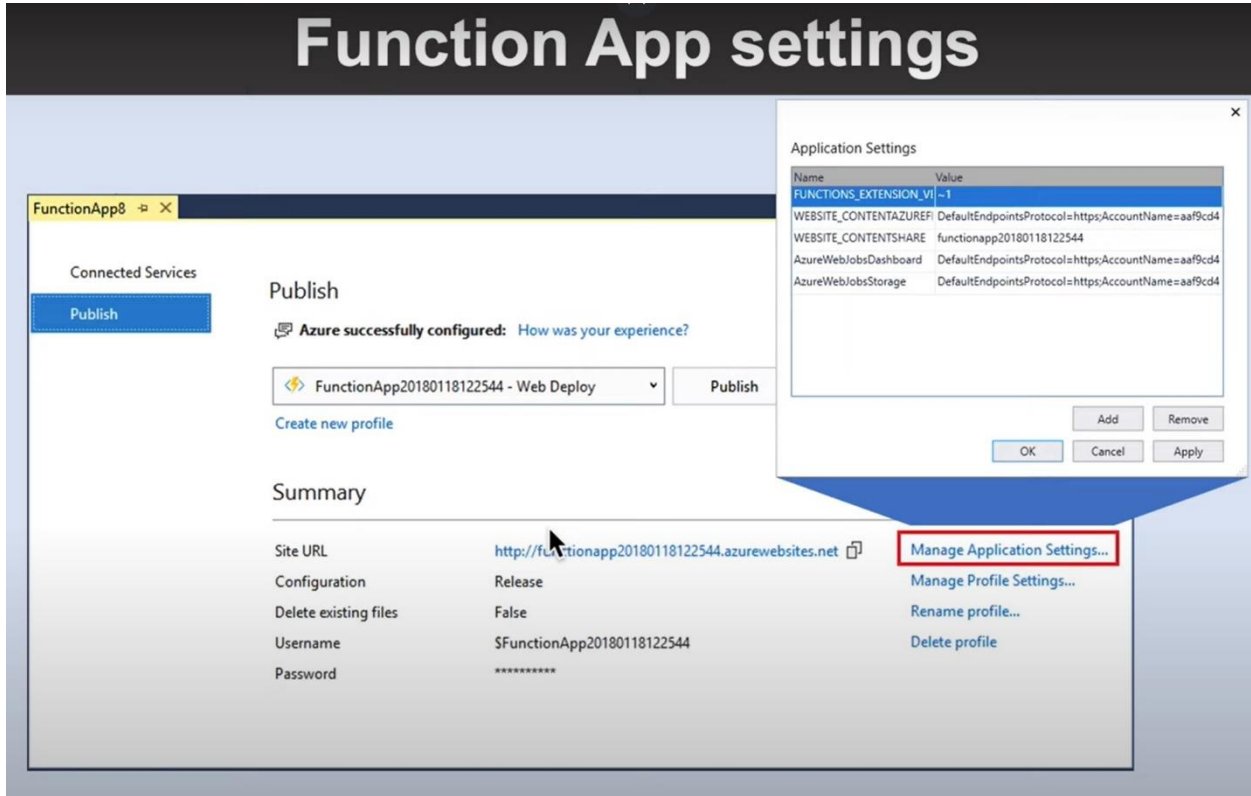
public static Person Run([QueueTrigger] JObject order, ILogger log)
{
    return new Person() {
        PartitionKey = "Orders",
        RowKey = Guid.NewGuid().ToString(),
        Name = order["Name"].ToString(),
        MobileNumber = order["MobileNumber"].ToString()
    };
}

```

Note: An orange callout bubble points to the [QueueTrigger] attribute, stating "Name of input parameter".

C#

Function App settings



FunctionApp8 [X]

Connected Services

Publish

Azure successfully configured: [How was your experience?](#)

FunctionApp20180118122544 - Web Deploy [v] **Publish**

[Create new profile](#)

Summary

Site URL	http://functionapp20180118122544.azurewebsites.net	Manage Application Settings...
Configuration	Release	Manage Profile Settings...
Delete existing files	False	Rename profile...
Username	SFunctionApp20180118122544	Delete profile
Password	*****	

Application Settings

Name	Value
FUNCTIONS_EXTENSION_V1	~1
WEBSITE_CONTENTAZURE	DefaultEndpointsProtocol=https;AccountName=aa19cd4
WEBSITE_CONTENTSHARE	functionapp20180118122544
AzureWebJobsDashboard	DefaultEndpointsProtocol=https;AccountName=aa19cd4
AzureWebJobsStorage	DefaultEndpointsProtocol=https;AccountName=aa19cd4

Buttons: Add, Remove, OK, Cancel, Apply

Durable Functions Chaining/Fan In – Fan Put

Durable Functions

- Write stateful functions in a stateless environment
- Manages state, checkpoints, and restarts
- Defines an Orchestrator function
 - Workflows are defined in code
 - Calls other functions synchronously or asynchronously
 - Checkpoint progress whenever function awaits

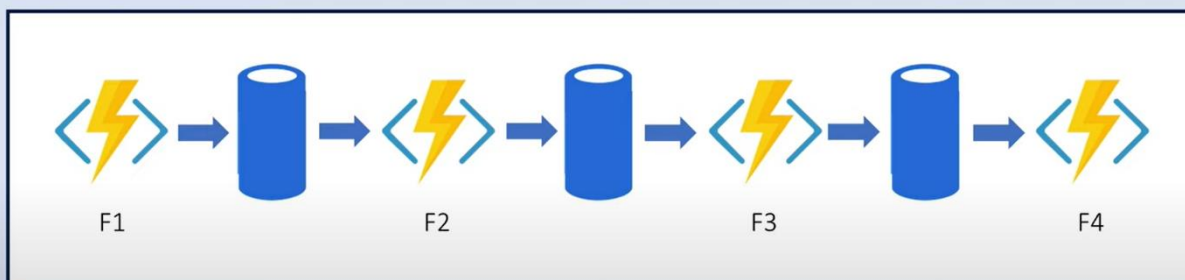
Durable Functions types

Orchestrator	Activity	Entity	Client
• Defines function workflows • Stateful	• The functions and tasks being orchestrated • Stateless	• Reads and updates small pieces of state • Stateful	• Sends messages to trigger Orchestrator and Entity functions • Stateless

- State is checkpointed and maintained in Azure Storage

Durable Function scenario - Chaining

Function chaining refers executing a sequence of functions in a particular order. Often, the output of one function needs to be applied to the input of another function.



Durable Function scenario - Chaining code

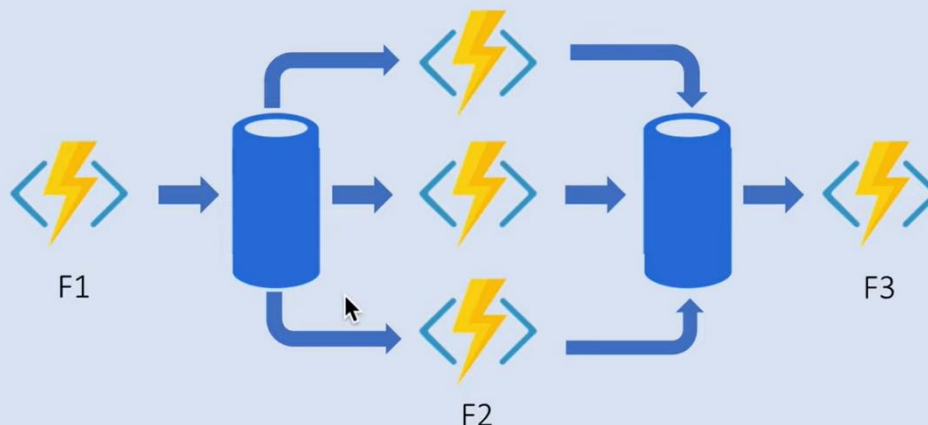
```
[FunctionName("Chaining")]
public static async Task<object> Run(
    [OrchestrationTrigger] IDurableOrchestrationContext context)
{
    try
    {
        var x = await context.CallActivityAsync<object>("F1", null);
        var y = await context.CallActivityAsync<object>("F2", x);
        var z = await context.CallActivityAsync<object>("F3", y);
        return await context.CallActivityAsync<object>("F4", z);
    }
    catch (Exception)
    {
        // Error handling or compensation goes here.
    }
}
```



Fan Out

Durable Function scenario - Fan-out/fan-in

Fan-out/fan-in refers to the pattern of executing multiple functions in parallel, and then waiting for all to finish



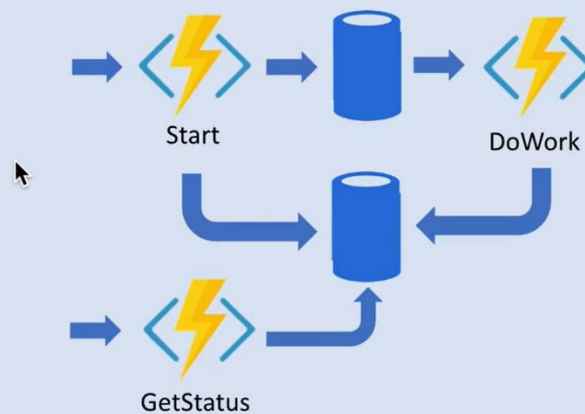
Fan In Requires Code

Durable Function scenario - Fan-out/fan-in code

```
[FunctionName("FanOutFanIn")]
public static async Task Run([OrchestrationTrigger] IDurableOrchestrationContext context)
{
    var parallelTasks = new List<Task<int>>();
    // Get a list of N work items to process in parallel.
    object[] workBatch = await context.CallActivityAsync<object[]>("F1", null);
    for (int i = 0; i < workBatch.Length; i++)
    {
        Task<int> task = context.CallActivityAsync<int>("F2", workBatch[i]);
        parallelTasks.Add(task);
    }
    await Task.WhenAll(parallelTasks);
    // Aggregate all N outputs and send the result to F3.
    int sum = parallelTasks.Sum(t => t.Result);
    await context.CallActivityAsync("F3", sum);
}
```

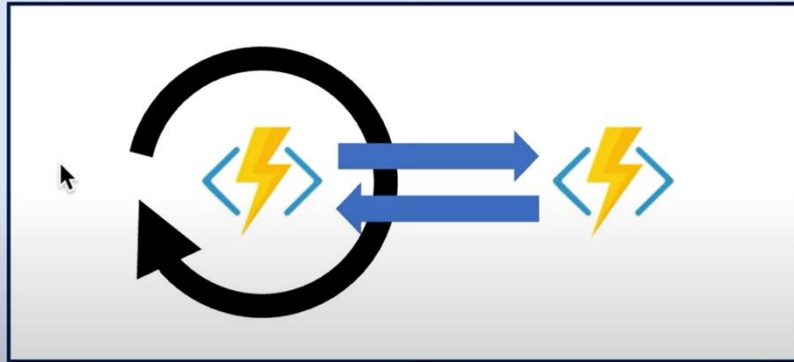
Durable Function scenario - Async HTTP APIs

Durable Functions provides built-in APIs that simplify the code that you write for interacting with long-running function executions



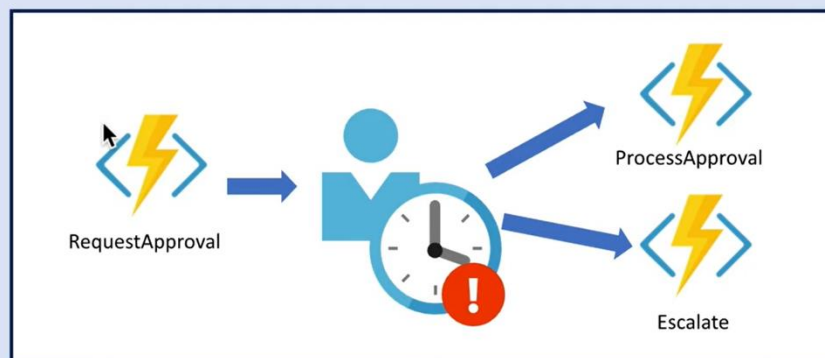
Durable Function scenario - Monitoring

The monitor pattern refers to a flexible recurring process in a workflow—for example, polling until certain conditions are met



Durable Function scenario - Human interaction

Many processes involve human interaction. Automated processes must allow for human low availability, and they often do so by using time-outs and compensation logic.



Why is it all Text?

JSON Object {} - columns rows - name:value pairs - key:value pairs vocabulary

array[] - zero based index - single unique or lists

null values - empty spots - default values

functions()

= assignment - creat a variable in RAM

Traverse the DOM - Document Object Model - search it getElementById - one to one

Plus sign

+

-number i get math $1 + 1 = 2$ - data math sum()

-string - concatenates does NOT do math - $1 + 1 = 11$

data persist not save

agent - scheduling tool that runs jobs & execute code (Azure Function)

Production

Datawarehouse - non nomalizes tables in database

Dimension

FactTable

-Code Bases

CMD - console apps

PowerShell - console apps

Bash - console apps

Az CLI - console apps

JavaScript

C#

GO

TypeScript

VB

DAX - Data Language Excel - Power BI Desktop - Power Query Editor

- Frameworks

Bootstrap

- Web Servers - made run in HTML

IIS - Msoft

Apache - Linux

maps Domain names https:myDomain.com - A Record - to file system

c:\inputpub\wwwroot\myDomain\MyApp - C# - Node.JS - VB

Execute Server Side Code

--- No Code Enviroment Azure

Office365 - Azure AD - PowerBI Service (Reports - DataSet) - Power Automate

MS Entra ID Formerly KA - Active Direct group of users - one to many relationship

Personal Accts - mrtimd@hotmail.com 2007 - live.com - outlook.com - gmail.com

Resource Groups - Resource - RBAC